Homework 4

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Pairwise Global Alignment

Regions of similarity in two genetic strings that are presumed to be homologous can be found by alignment. As shown in Figure 1, a global alignment of two strings is formed by adding gap symbols ('-') to each string to make them the same length so that every symbol in one string corresponds to a symbol in the other. A gap symbol in an aligned string represents a deletion at that position or an insertion in the string it is aligned to. Mismatches can be interpreted as point mutations that change symbols at single positions in the strings.

```
AAB24882        TYHMCQFHCRYVNNHSGEKLYECNERSKAFSCPSHLQCHKRRQIGEKTHEHNQCGKAFPT 60
AAB24881        --------------------YECNQCGKAFAQHSSLKCHYRTHIGEKPYECNQCGKAFSK 40
                                    ****: .***:  * *:** * :****.:* ********..

AAB24882        PSHLQYHERTHTGEKPYECHQCGQAFKKCSLLQRHKRTHTGEKPYE-CNQCGKAFAQ- 116
AAB24881        HSHLQCHKRTHTGEKPYECNQCGKAFSQHGLLQRHKRTHTGEKPYMNVINMVKPLHNS 98
                 **** *:***********:***:**.: .***************    :  *.: :
```

Figure 1: An alignment of two human zinc finger proteins, identified by their GenBank accession numbers on the left. A '*' below aligned symbols denotes identical amino acids, a ':' denotes similar amino acids, a '.' denotes less similar amino acids, and a blank space denotes dissimilar amino acids. Color-coding represents the amino acid's chemical properties (e.g., blue denotes acidic and green denotes basic).

There are many possible alignments for a pair of strings. In practice, we cannot know which is the absolute "best" alignment of two genetic strings. This is because discriminating between alignments requires assumptions about how the strings have evolved, and this information about their evolution is precisely what we hope to learn from the alignment. The best we can do is to compare alignments quantitatively by using a scoring approach. Many such alignment scores assign 0 to a match, a negative number to a mismatch, and an even larger negative number to a gap. Summing all these scores over an alignment gives the alignment's score; in keeping with the assumption of parsimony. The optimal alignment will have the highest score, which implies the fewest possible changes between the strings.

A scoring matrix contains a score for matches and mismatches between each pair of symbols. These scores are derived empirically based on the frequency with which these matches and mismatches occur in highly similar biological sequences for which the true alignment is practically unequivocal.

A gap penalty is the score deducted for the presence of a gap. One of the most commonly used gap penalties is an affine gap penalty, in which one penalty is assigned for adding the first symbol to a gap, and a different penalty is charged for every additional symbol in the gap. The gap opening penalty is usually larger than the gap extension penalty because a single deletion of k nucleotides is more likely than k independent, contiguous, single-nt deletions.

For example, say that we use a simple scoring matrix that rewards +1 for all matching symbols and charges -1 for all mismatches, along with an affine gap penalty that has a gap opening penalty of -5 and a gap extension penalty of -1. The following alignment has 14 matched symbols, 3 mismatches, and two gaps (one of which is extended by three symbols), which gives a total score for the alignment of $14 - 3 - (2 \cdot 5 + 3) = -2$.

```
GARFIELDTHEVERYFA-TCAT
:::|||||||||    || ||||
WINFIELDTHE----FASTCAT
```

EBI hosts many different online user interfaces for bioinformatics tools, including Needle and Stretcher. Needle and Stretcher are tools from the EMBOSS package that are for aligning genetic strings. Both can be accessed from this website.

The programs are very similar. One difference is that Stretcher always uses an end gap penalty, while Needle includes the option but does not use it by default. In either of these interfaces, you can enter strings directly or in any supported format: GCG, FASTA, EMBL, GenBank, PIR/NBRF, Phylip, or the UniProtKB/Swiss-Prot format. Feel free to play around with these interfaces by entering pairs of strings. Notice that clicking on More options... under "Step 2" will allow you to change the alignment parameters as well as the output format.

PROBLEM

An online interface to EMBOSS's Needle tool for aligning DNA and RNA strings can be found here.

Use: the following settings:

- The DNAfull scoring matrix.
- Gap opening penalty of 10.
- Gap extension penalty of 1.
- End gap set to true.
- End gap opening penalty of 10.
- End gap extension penalty of 1.

For our purposes, the "pair" output format will work fine; this format shows the two strings aligned at the bottom of the output file beneath some statistics about the alignment, including the global alignment score.

Given: A file containing two GenBank IDs.

Return: The maximum global alignment score between the DNA strings associated with these IDs.

Note: instead of code, please submit a screenshot of the Needle Tool Output webpage.

Sample Dataset

```
JX205496.1 JX469991.1
```

Sample Output

```
257
```

Hint: You can download the EMBOSS suite and run Needle and Stretcher locally via the command line.

2. Suboptimal Local Alignment

Different regions of the genome evolve in different ways. In coding regions, where any significant change can be lethal to the organism, the most common source of variation is point mutations. In non-coding regions (introns, non-coding RNA, or intergenic spacers), we find a different situation: entire intervals can easily be duplicated, inserted, deleted, or reversed.

During the 1940s and 1950s, Barbara McClintock discovered the effect of genetic transposition in maize plants and demonstrated that many genome rearrangements are caused by regions called transposons, or relatively short intervals of DNA that can change their location within the genome. For the discovery of these "jumping genes," McClintock was awarded the Nobel prize for Physiology or Medicine in 1983.

Figure 1 illustrates the mechanism of transposon duplication during recombination. In this figure, two transposons are turned into three, and any genes located between the two original occurrences of the transposon will be duplicated. This process can also work in the opposite direction to delete genes. Both duplication and deletion can prove harmful to the organism, although deletion is more likely to be harmful.

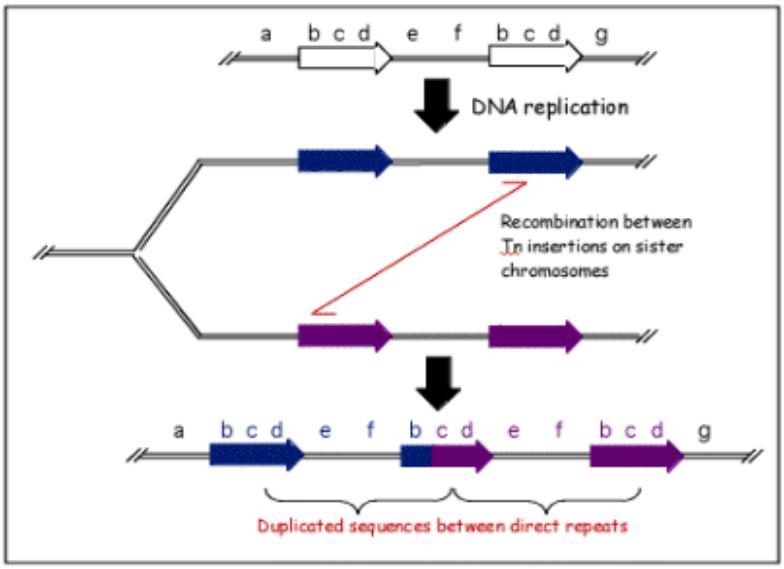

Figure 1: An illustration of a specific type of transposon called a Tn element, which recombines with a copy of itself farther down the chromosome during meiosis. As a result, the region between the transposon becomes duplicated, and an additional Tn element is formed. ©Stanley Maloy, University of California, Irvine.

As with any genomic region, transposons may undergo point mutations and wind up not being completely identical: they only need to be similar enough to facilitate recombination. However, you cannot find multiple repeats in the strings by using the common method for local alignment, as it will find only a single best match.

For example, if you try to locally align the strings "AAAATTTTT" and "TTTTTAAAA", the following optimal local alignment will be returned:

```
001    1 TTTTT 5
         |||||
002    5 TTTTT 9
```

To find multiple similar regions in two genetic strings simultaneously, we will need to apply a technique called suboptimal alignment, which can obtain more than one different local alignment. When aligning "AAAATTTTT" and "TTTTTAAAA", we should also find the region "AAAA" of similarity:

```
001    6 AAAA 9
         ||||
002    1 AAAA 4
```

To obtain multiple alternative matches via suboptimal alignment, we can use the program **Lalign**. **Lalign** has input and output parameters like those of **Needle** and **Water** (the "+5/-4" matrix is equivalent to DNAfull), but the user can limit the number of alignments reported by changing the "expectation threshold", which is the maximum number of times a match is expected to occur by random chance. A higher expectation threshold will return more disjoint regions of similarity in the two input strings.

PROBLEM

The **Lalign** program for finding multiple alternative matches via suboptimal alignment is available here.

Given: A file containing two DNA strings s and t in FASTA format that share some short inexact repeat r of 32-40 bp. By "inexact" we mean that r may appear with slight modifications (each repeat differ by ≤ 3 changes/indels).

Return: The total number of occurrences of r as a substring of s, followed by the total number of occurrences of r as a substring of t.

Note: instead of code, please submit a screenshot of the Lalign Tool Output webpage.

Sample Dataset

```
>Rosalind_12
GACTCCTTTGTTTGCCTTAAATAGATACATATTTACTCTTGACTCTTTTGTTGGCCTTAAATAGATACATATTTGTGCGACTCCACGAGTGAT
TCGTA
>Rosalind_37
ATGGACTCCTTTGTTTGCCTTAAATAGATACATATTCAACAAGTGTGCACTTAGCCTTGCCGACTCCTTTGTTTGCCTTAAATAGATACATAT
TTG
```

Sample Output

```
2 2
```

Hint: The problem is difficult to solve by running a single local alignment. After the first alignment of the two input sequences, search for the r string. Once you have identified the repeat sequence r, you can then align the repeat sequence to each of the input strings to solve the problem.

Lalign is part of William Pearson's FASTA package. You can download the program and run it locally via the command line.

3. Counting Point Mutations

A mutation is simply a mistake that occurs during the creation or copying of a nucleic acid, in particular DNA. Because nucleic acids are vital to cellular functions, mutations tend to cause a ripple effect throughout the cell. Although mutations are technically mistakes, a very rare mutation may equip the cell with a beneficial attribute. In fact, the macro effects of evolution are attributable to the accumulated result of beneficial microscopic mutations over many generations.

The simplest and most common type of nucleic acid mutation is a point mutation, which replaces one base with another at a single nucleotide. In the case of DNA, a point mutation must change the complementary base accordingly; see Figure 1.

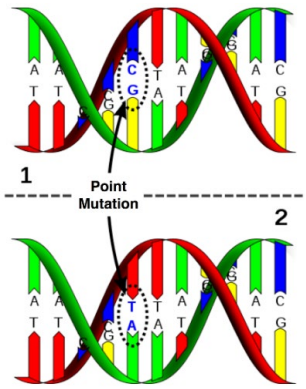

Figure 1: A point mutation in DNA changing a C-G pair to an A-T pair.

In biology, the adjective "homologous" is applied to two organisms or structures that share a (recent) common ancestor. Traditionally, the term has been applied to physical comparison (e.g., using the fossil record), but in the modern era we may commonly see nucleic acids or polypeptides of similar function being described as "homologous". Two DNA strands taken from different organism or species genomes are homologous if they share a recent ancestor; thus, counting the number of bases at which homologous strands differ provides us with the minimum number of point mutations that could have occurred on the evolutionary path between the two strands.

We are interested in minimizing the number of (point) mutations separating two species because of the biological principle of parsimony, which demands that evolutionary histories should be as simply explained as possible.

PROBLEM

For two strings s and t of equal length, the Hamming distance between s and t, denoted $d_H(s,t)$, is the number of corresponding symbols that differ in s and t. See Figure 2.

GAGCCTACTAACGGGAT
CATCGTAATGACGGCCT

Figure 2: The Hamming distance between these two strings is 7. Mismatched symbols are colored red.

Given: A file containing two DNA strings s and t of equal length (not exceeding 1 kb).

Return: The Hamming distance $d_H(s,t)$.

Sample Dataset

```
GAGCCTACTAACGGGAT
CATCGTAATGACGGCCT
```

Sample Output

```
7
```

4. Edit Distance

In the "Counting Point Mutations" problem, the calculated Hamming distance gave you a preliminary notion of the evolutionary distance between two DNA strings by counting the minimum number of single nucleotide substitutions that could have occurred on the evolutionary path between the two strands.

However, in practice, homologous strands of DNA or protein are rarely the same length because point mutations also include the insertion or deletion of a single nucleotide (and single amino acids can be inserted or deleted from peptides). Thus, you need to incorporate these insertions and deletions into the calculation of the minimum number of point mutations between two strings. One of the simplest models charges a unit "cost" to any single-symbol insertion/deletion, then (in keeping with parsimony) requests the minimum cost over all transformations of one genetic string into another by point substitutions, insertions, and deletions.

PROBLEM

For two strings s and t (of possibly different lengths), the edit distance $d_E(s,t)$ is the minimum number of edit operations needed to transform s into t, where an edit operation is defined as the substitution, insertion, or deletion of a single symbol.

The latter two operations incorporate the case in which a contiguous interval is inserted into or deleted from a string; such an interval is called a gap. For the purposes of this problem, the insertion or deletion of a gap of length k still counts as k distinct edit operations.

Given: A file containing two protein strings s and t in FASTA format (with each string having a length at most 1000 aa).

Return: The edit distance $d_E(s,t)$.

Sample Dataset

```
>Rosalind_39
PLEASANTLY
>Rosalind_97
MEANLY
```

Sample Output

```
5
```

Hint: Calculating the edit distance be done with dynamic programming. First, determine the number of edits to make string $S[:i]$ into $T[:j]$, and then move to $i+1$ or $j+1$.

The first row (turning $S[:i]$ into an empty string) is merely the length of $S[:i]$, or i. The same applies to the first column. From here we can determine the entry: $(i,j) = \min\{(i-1,j)+1, (i,j-1)+1, (i-1,j-1)+a\}$, where $a = 0$ if $S[i-1] = T[j-1]$, otherwise $a = 1$. Here, the first value corresponds to a deletion, the second to an insertion, and the third to either substitution or nothing.

Pseudo code

```
import copy
n=len(S)
m=len(T)
D=[[]]
temp=[0]*(n+1)
for k in range (n+1):D[0].append(k)
for k in range (m):
    temp[0]+=1
    D.append(copy.copy(temp))
for y in range (1,m+1):
    for x in range (1,n+1):
        if T[y-1]==S[x-1]:a=0
        else:a=1
        D[y][x]=min(D[y-1][x]+1,D[y][x-1]+1,D[y-1][x-1]+a)
print(D[m][n])
```

The algorithm requires $O(nm)$ time where n is the length of S and m is the length of T. You can conserve space by noticing that for every iteration you only use at most 2 rows of the 2d array. Optimizing your code can transform the space complexity from $O(nm)$ to $O(m)$ or $O(n)$.